Intelligent Systems,
MIPT

Natural Language Processing

L4 / 03.03.26

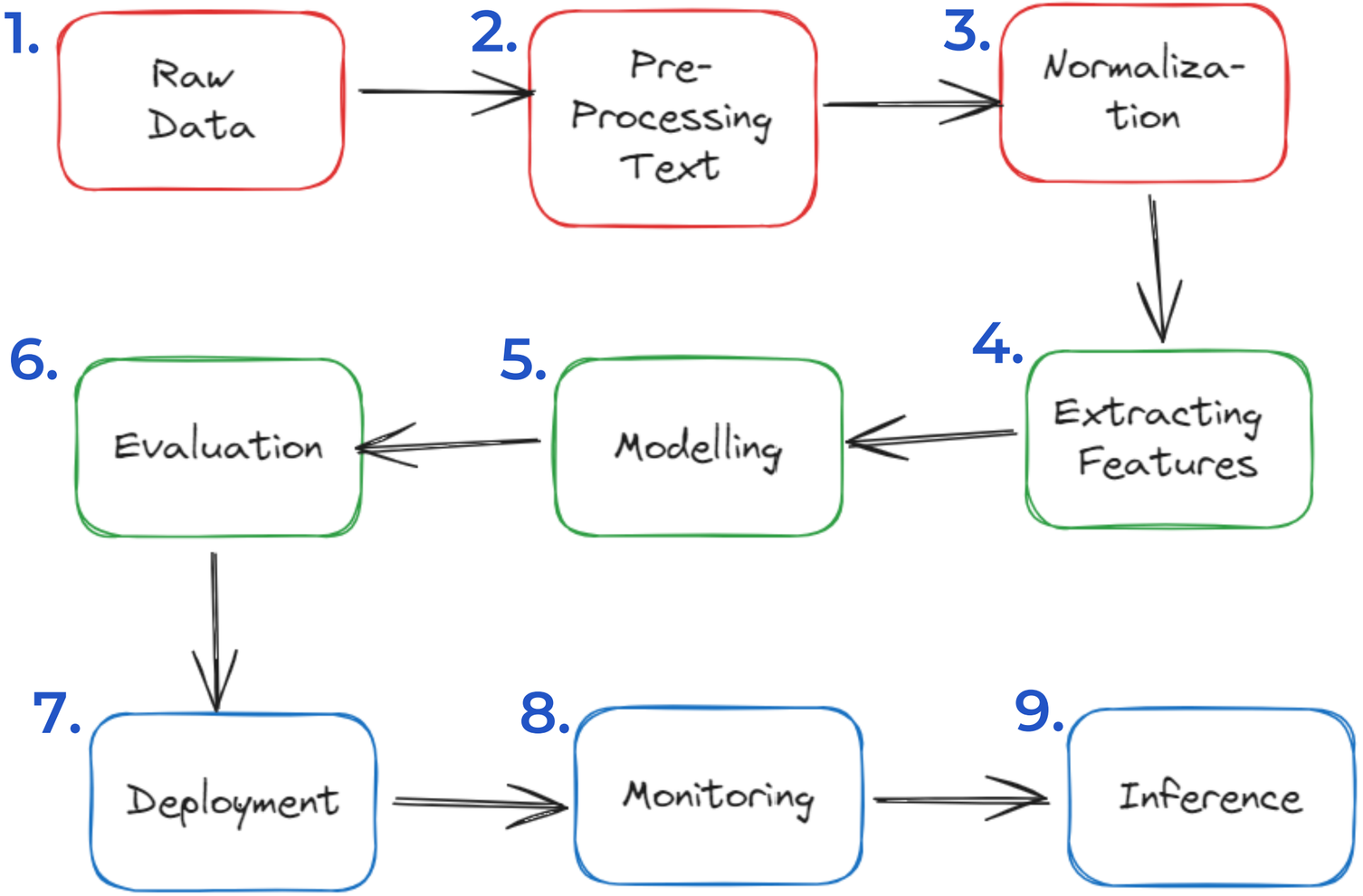
- Course page
- GitHub

Content

Lecture 4: Transfer Learning, BERT-like, LLMs

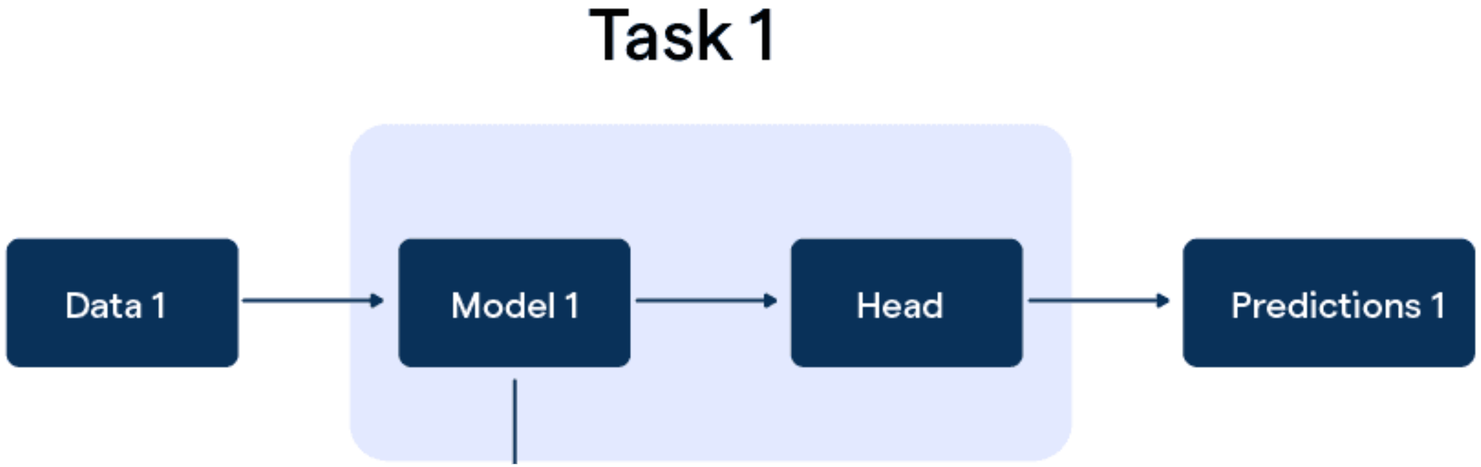
- Transfer Learning
- Pre-training paradigms, BERT, GPT, T5
- LLMs, Prompting

NLP pipeline



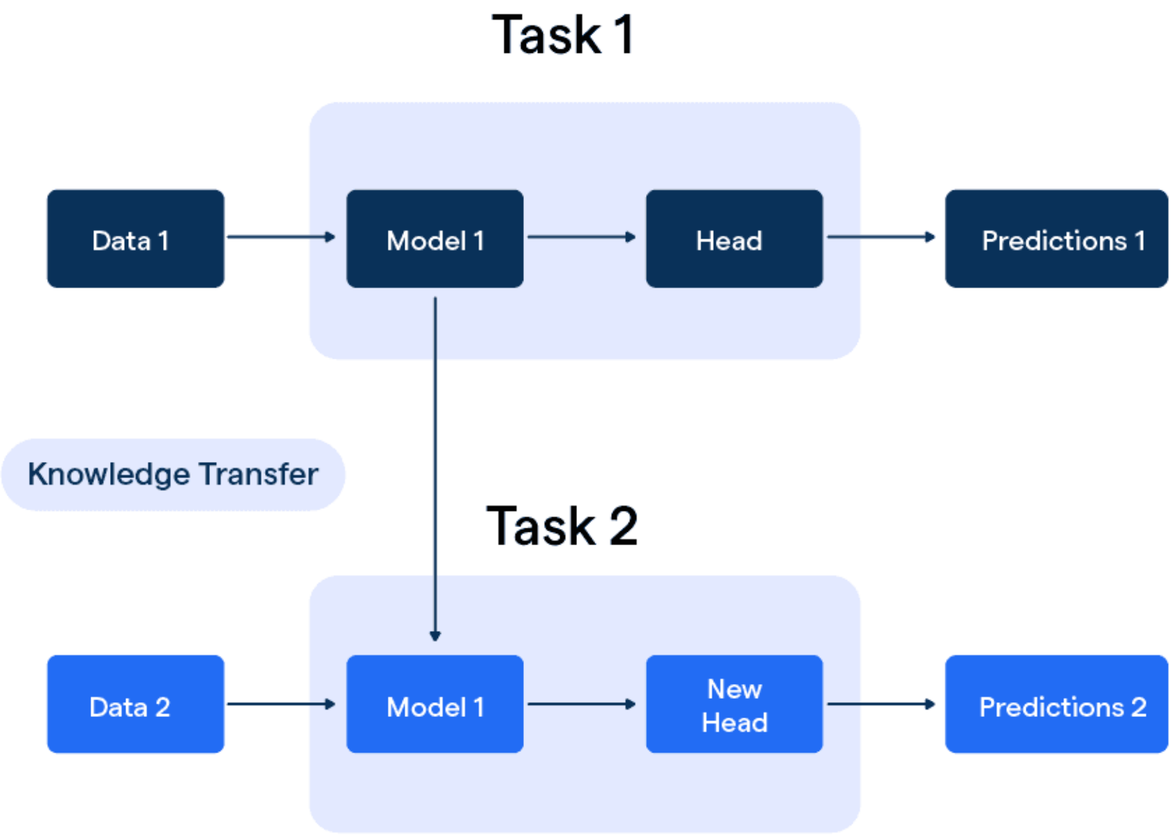
Transfer learning: intro

- **Traditional ML:** isolated, single task learning. Knowledge is not retained or accumulated. Learning is performed w/o considering past learned knowledge in other tasks



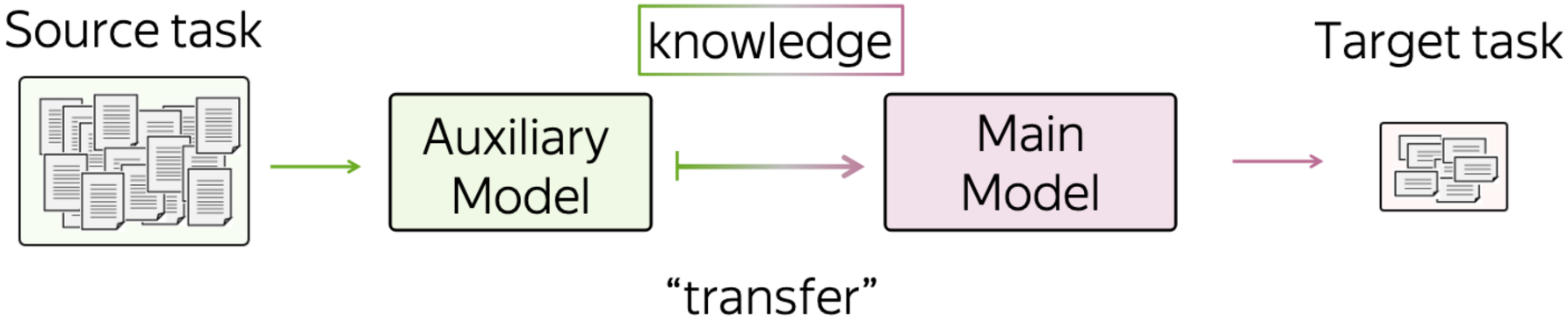
Transfer learning: intro

- **Transfer learning:** learning of a new tasks relies on the previous learned tasks. Learning process can be faster, more accurate and/or need less training data

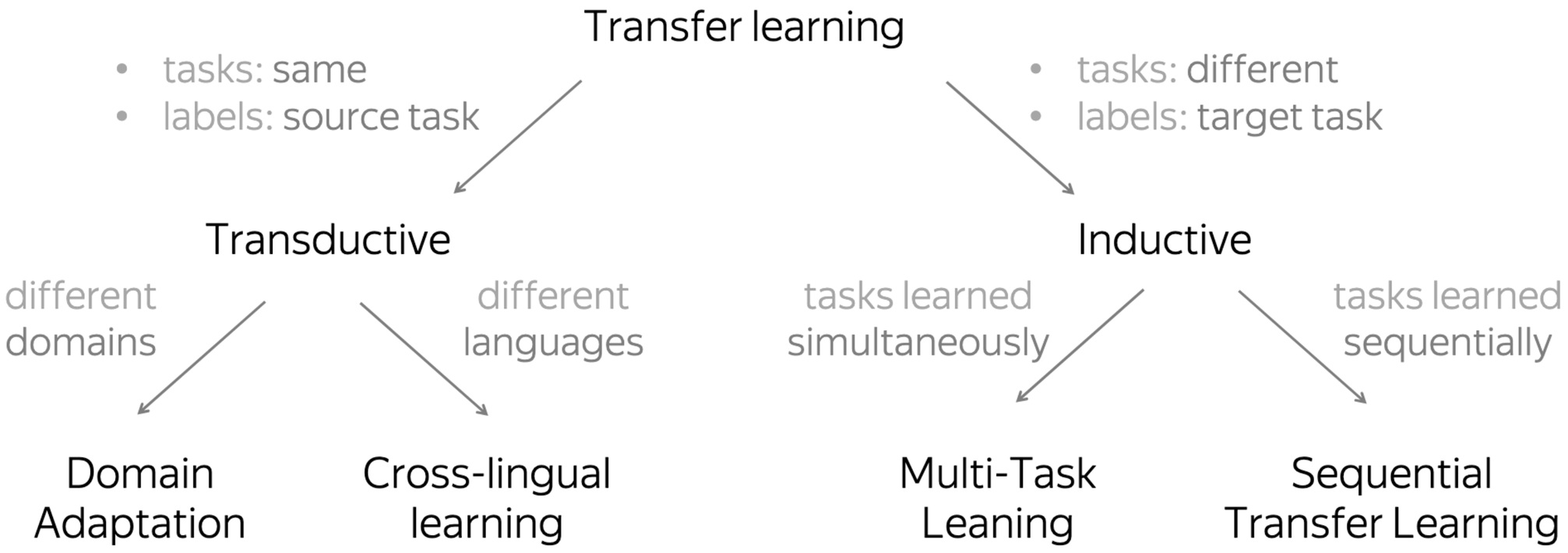


Transfer learning

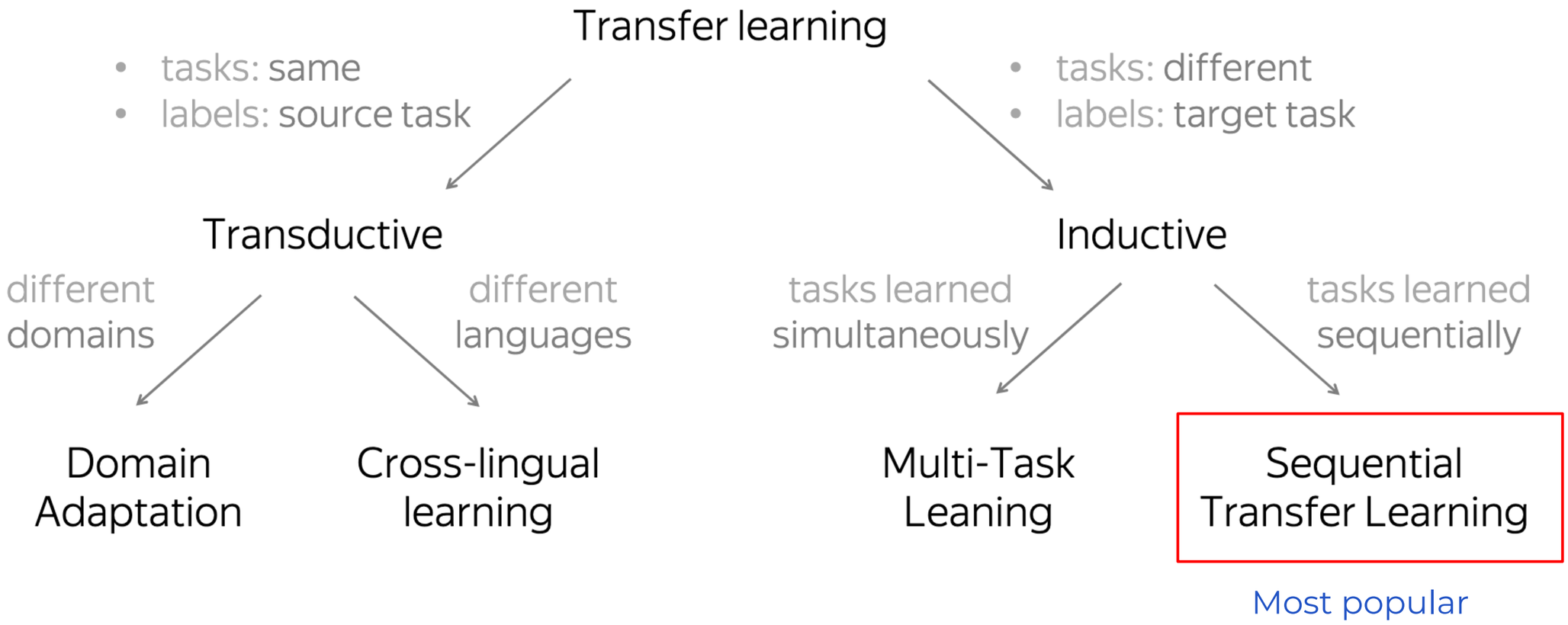
- Transfer learning uses what has been learned in one setting to improve generalization in another setting



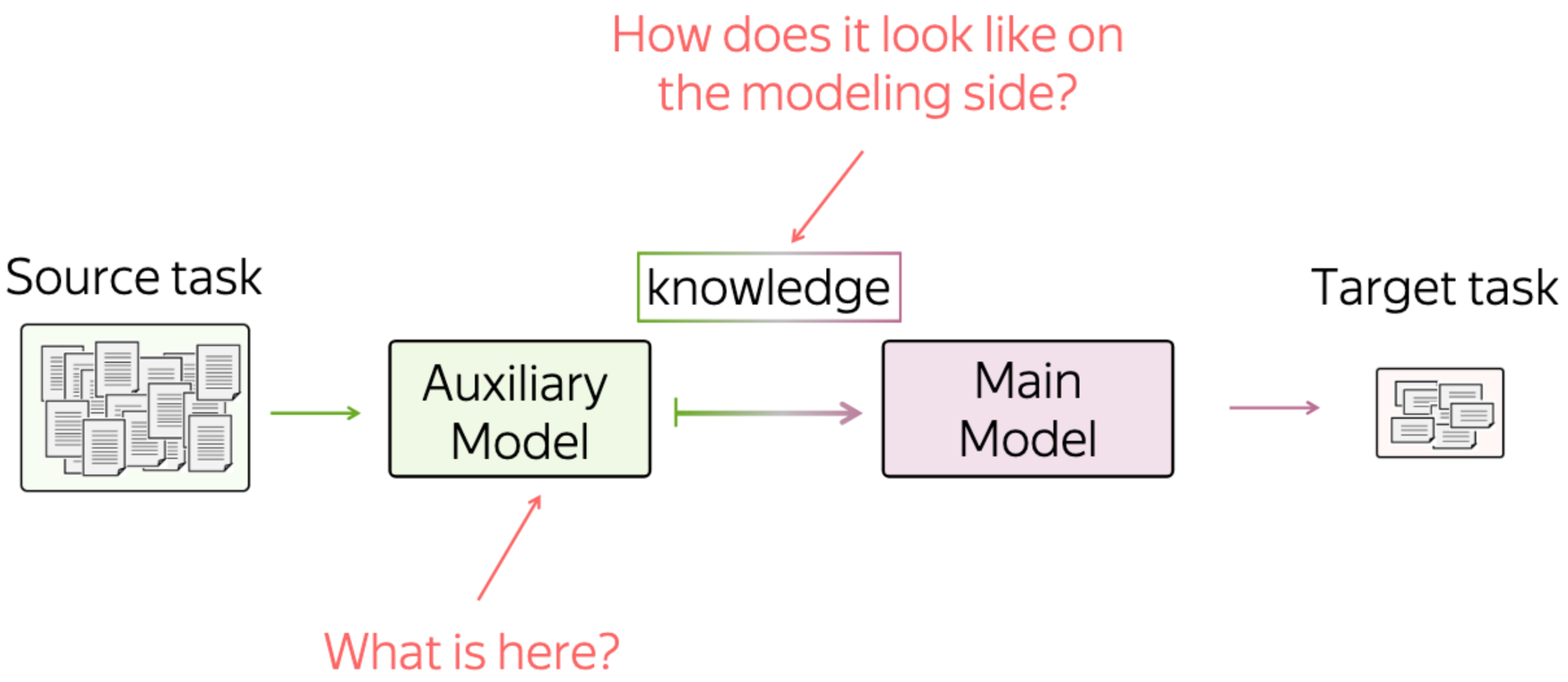
Transfer learning: taxonomy



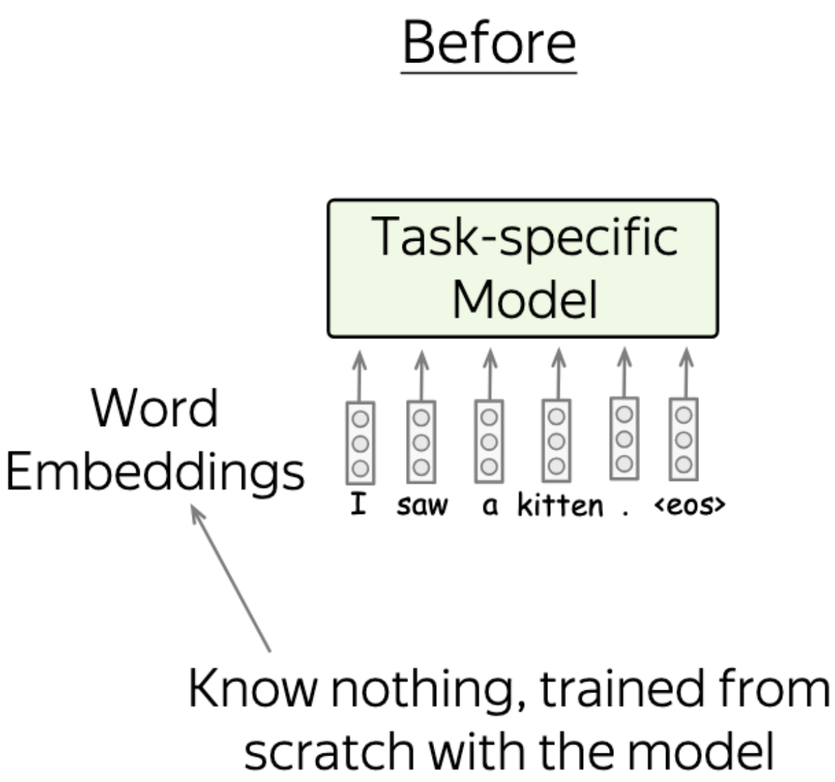
Transfer learning: taxonomy



Sequential transfer learning

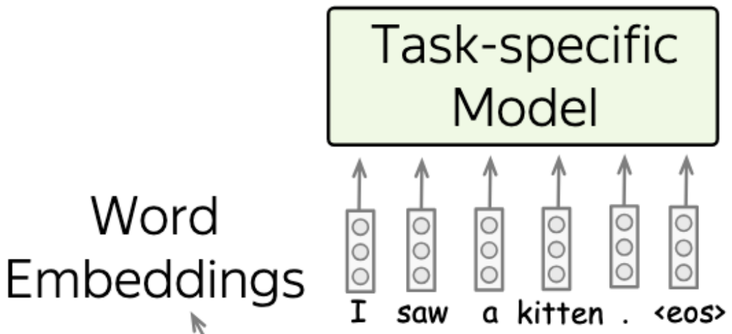


TL via Embeddings



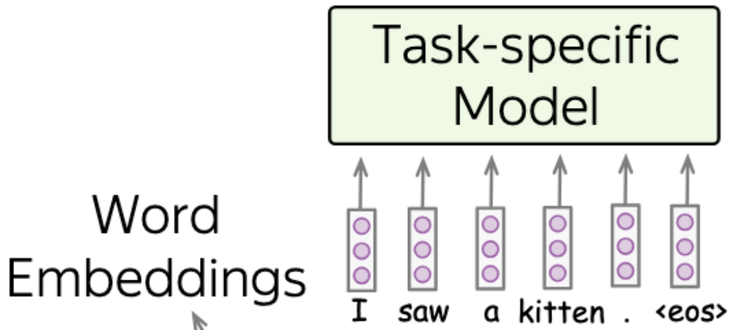
TL via Embeddings

Before



Know nothing, trained from scratch with the model

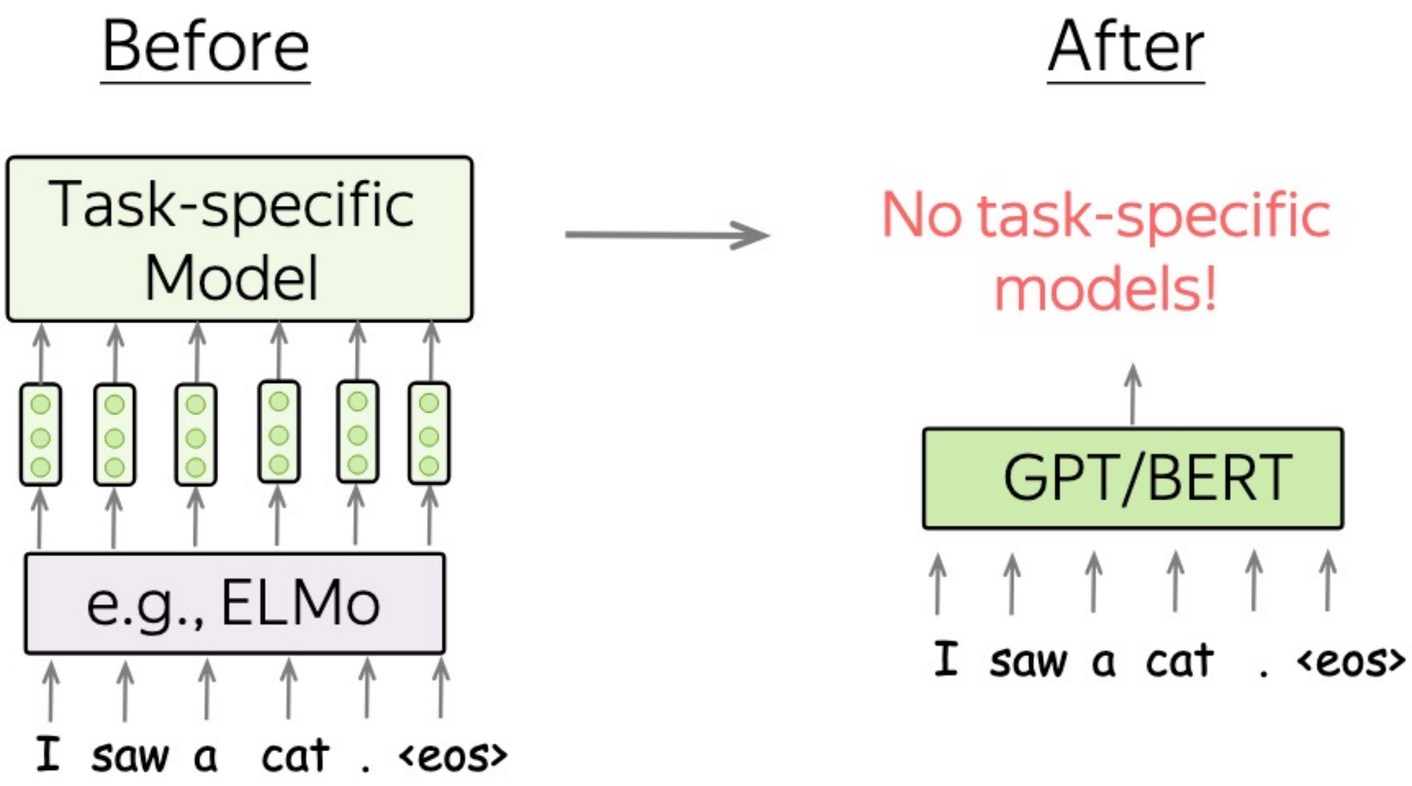
After



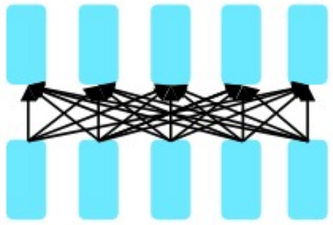
Word2Vec, GloVe, ... - saw a lot of data, know a lot

Transfer learning: general

- **Idea:** instead of replacing word embeddings, replace entire models.



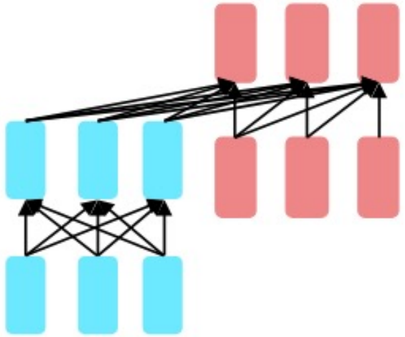
Pretraining architectures



Encoders

> Gets bidirectional context – can condition on future!

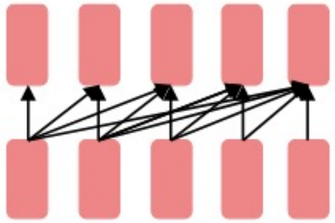
> How do we train them to build strong representations?



Encoder-Decoders

> Good parts of decoders and encoders?

> What's the best way to pretrain them?



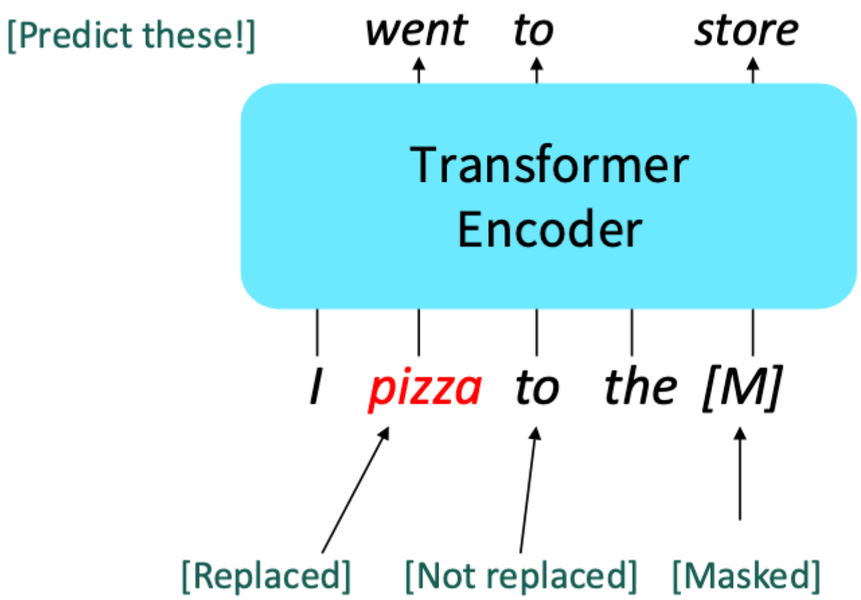
Decoders

> Language models! What we've seen so far.

> Nice to generate from; can't condition on future words

Pretraining encoders: BERT

- Some more details about Masked LM for BERT (Bidirectional Encoder Representations from Transformers)



Pretraining encoders: BERT

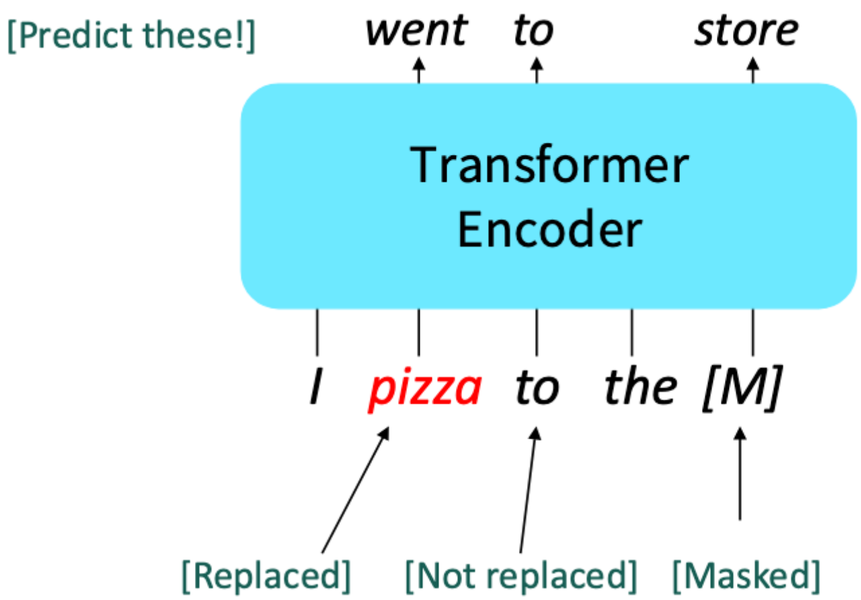
- **Some more details about Masked LM for BERT (Bidirectional Encoder Representations from Transformers)**

  > Predict a random 15% of (sub)word tokens:

  >> Replace input word with [MASK] 80% of the time

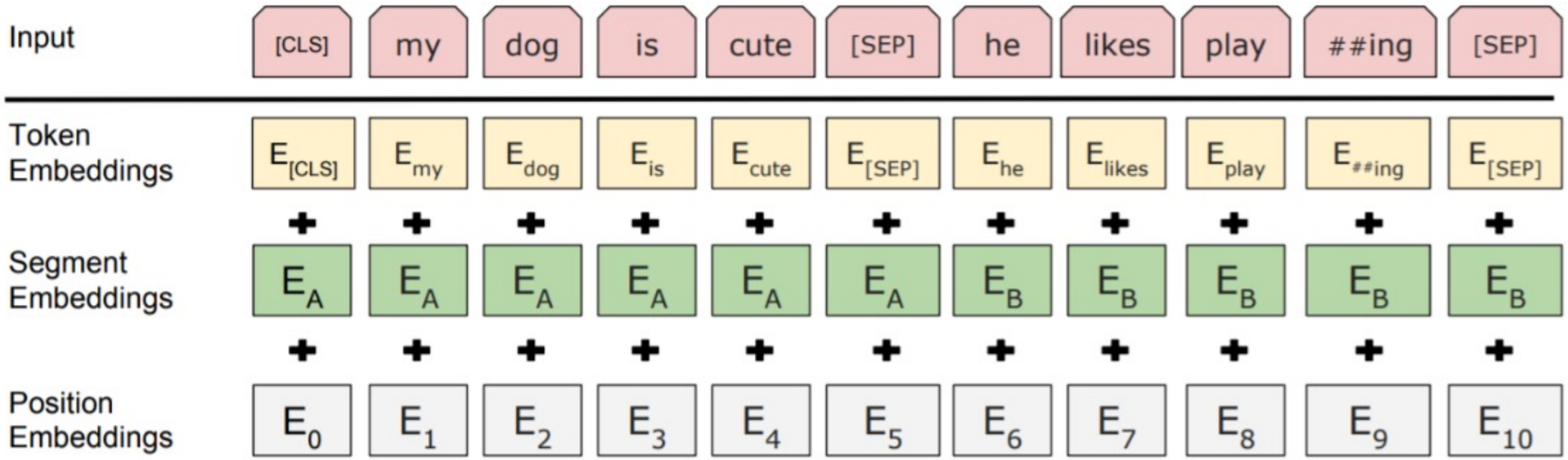
  >> Replace input word with a random token 10% of the time

  >> Leave input word unchanged 10% of the time (but still predict it!)



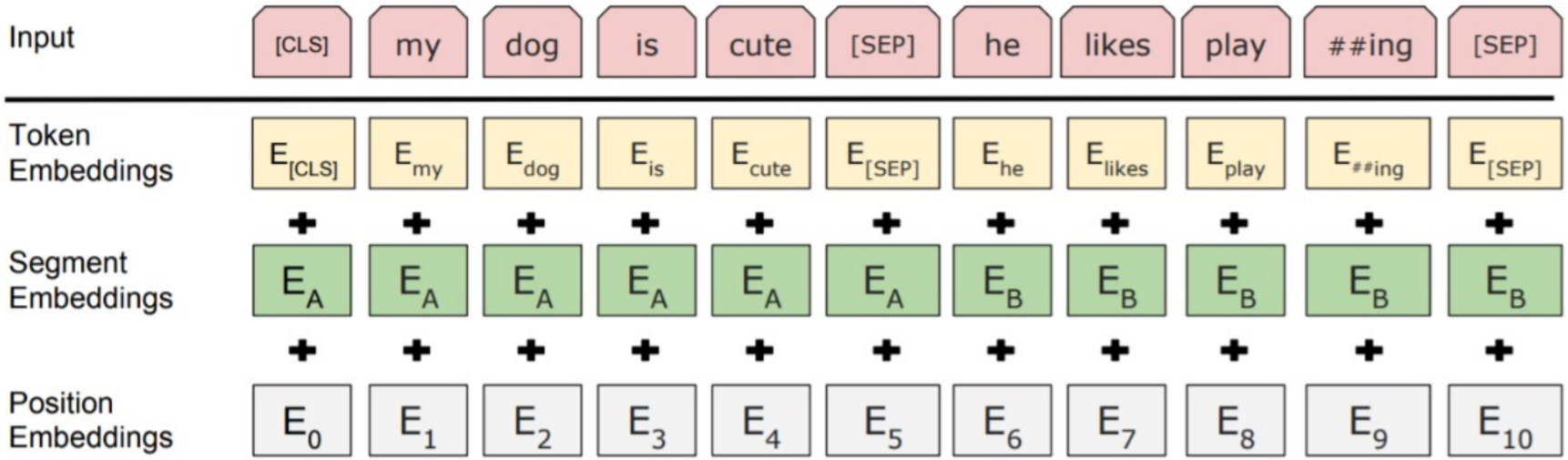
Pretraining encoders: BERT

- The pretraining input to BERT was two separate contiguous chunks of text:



Pretraining encoders: BERT

- The pretraining input to BERT was two separate contiguous chunks of text:



- BERT was trained to predict whether one chunk follows the other or is randomly sampled. Later work has argued this “next sentence prediction” is not necessary.

BERT: details

- **Two models were released:**
  - > BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
  - > BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.

BERT: details

- **Two models were released:**
  - > BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
  - > BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- **Trained on:**
  - > BooksCorpus (800 million words)
  - > English Wikipedia (2,500 million words)

BERT: details

- **Two models were released:**
  - > BERT-base: 12 layers, 768-dim hidden states, 12 attention heads, 110 million params.
  - > BERT-large: 24 layers, 1024-dim hidden states, 16 attention heads, 340 million params.
- **Trained on:**
  - > BooksCorpus (800 million words)
  - > English Wikipedia (2,500 million words)
- **Pretraining is expensive and impractical on a single GPU.**
  - > BERT was pretrained with 64 TPU chips for a total of 4 days.
  - > (TPUs are special tensor operation acceleration hardware)
- **Finetuning is practical and common on a single GPU**

BERT: details

- **BERT was massively popular and hugely versatile; finetuning this model led to new SOTA results on a broad range of tasks**

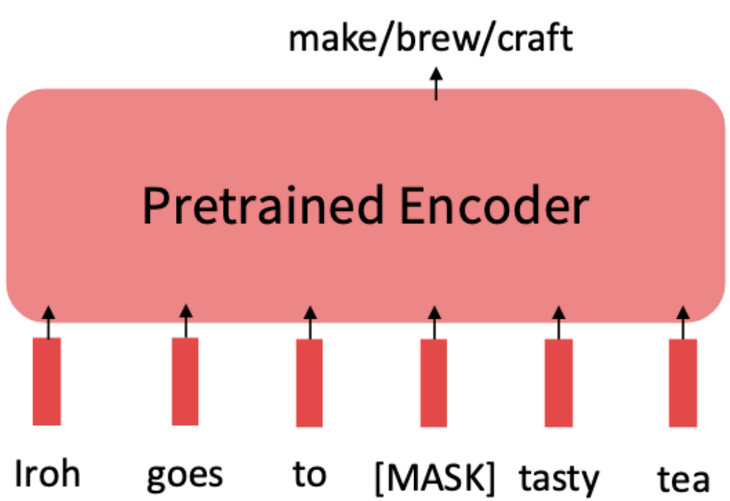
  > QQP: Quora Question Pairs (detect paraphrase questions)
  > QNLI: natural language inference over question answering data
  > SST-2: sentiment analysis
  > CoLA: corpus of linguistic acceptability (detect whether sentences are grammatical.)
  > STS-B: semantic textual similarity
  > MRPC: microsoft paraphrase corpus
  > RTE: a small natural language inference corpus

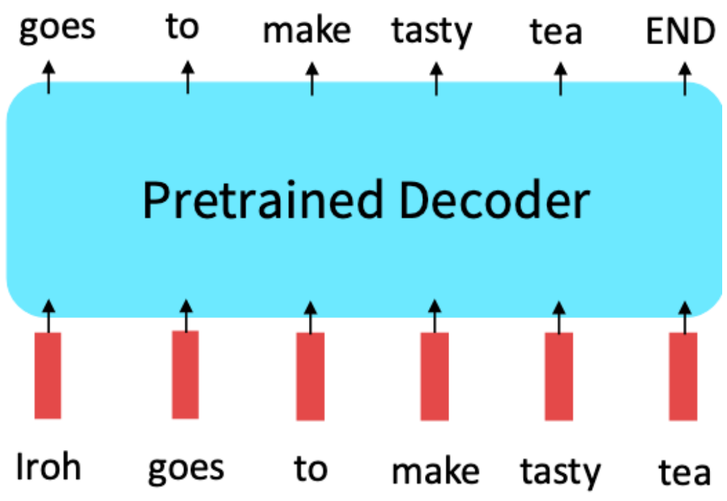
System	MNLI-(m/mm)	QQP	QNLI	SST-2	CoLA	STS-B	MRPC	RTE	**Average**
	392k	363k	108k	67k	8.5k	5.7k	3.5k	2.5k	-
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

Limitations of encoder

- **Why not used pretrained encoders for everything?**

  > If your task involves generating sequences, consider using a pretrained decoder; BERT and other pretrained encoders don't naturally lead to nice autoregressive (1-word-at-a time) generation methods.



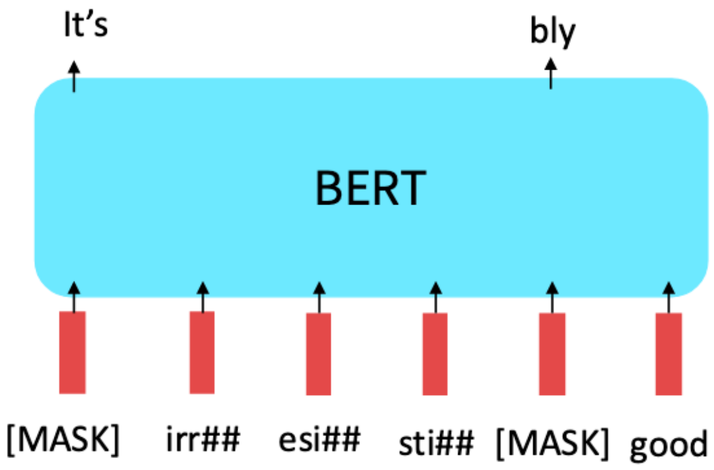


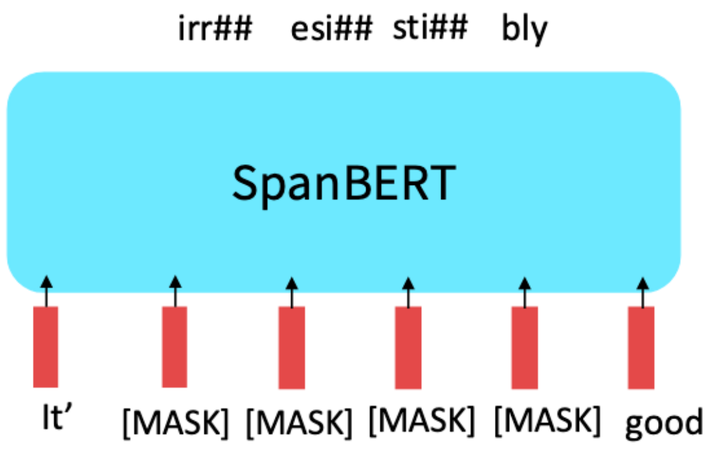
Extensions of encoder

- **You'll see a lot of BERT variants like RoBERTa, XLM, SpanBERT, ...**

  > Some generally accepted improvements to the BERT pretraining formula:

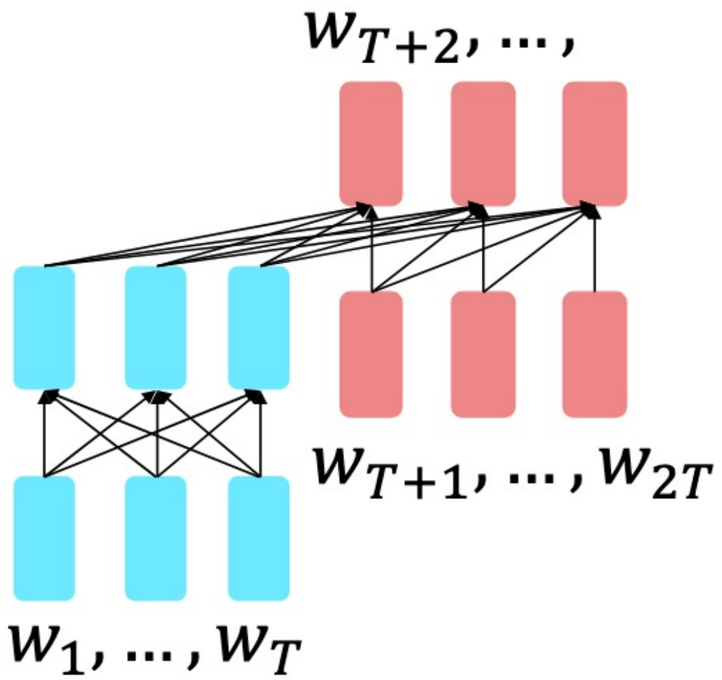
  >> RoBERTa: mainly just train BERT for longer and remove next sentence prediction!
  >> SpanBERT: masking contiguous spans of words makes a harder, more useful pretraining task
  >> ModernBERT: ???





Encoder-decoder: objective

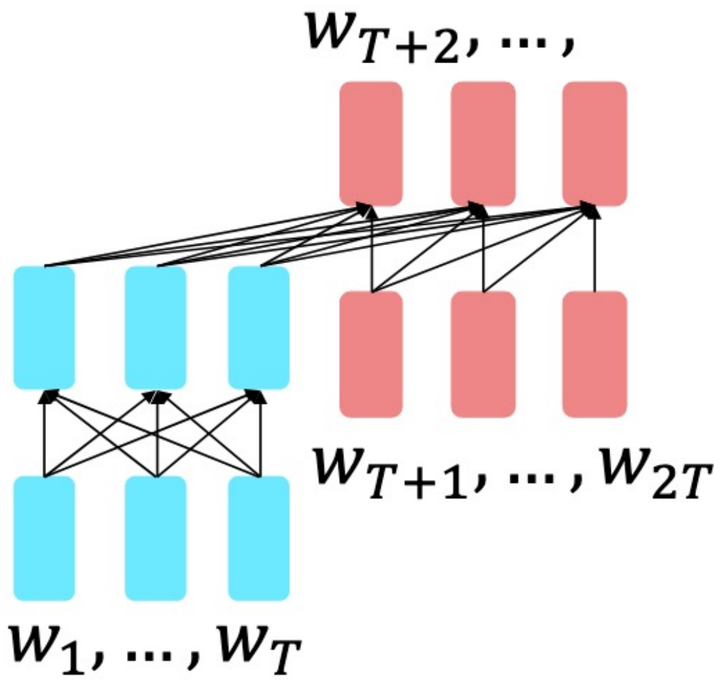
- For encoder-decoders, we could do something like language modeling, but where a prefix of every input is provided to the encoder and is not predicted.



Encoder-decoder: objective

- For encoder-decoders, we could do something like language modeling, but where a prefix of every input is provided to the encoder and is not predicted.

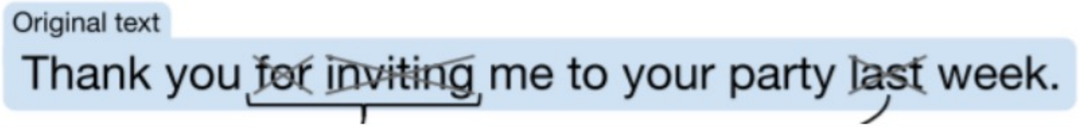
$$h_1, \ldots, h_T = \text{Encoder}(w_1, \ldots, w_T)$$
$$h_{T+1}, \ldots, h_2 = Decoder(w_1, \ldots, w_T, h_1, \ldots, h_T)$$
$$y_i \sim Ah_i + b, i > T$$



- The encoder portion benefits from bidirectional context; the decoder portion is used to train the whole model through language modeling

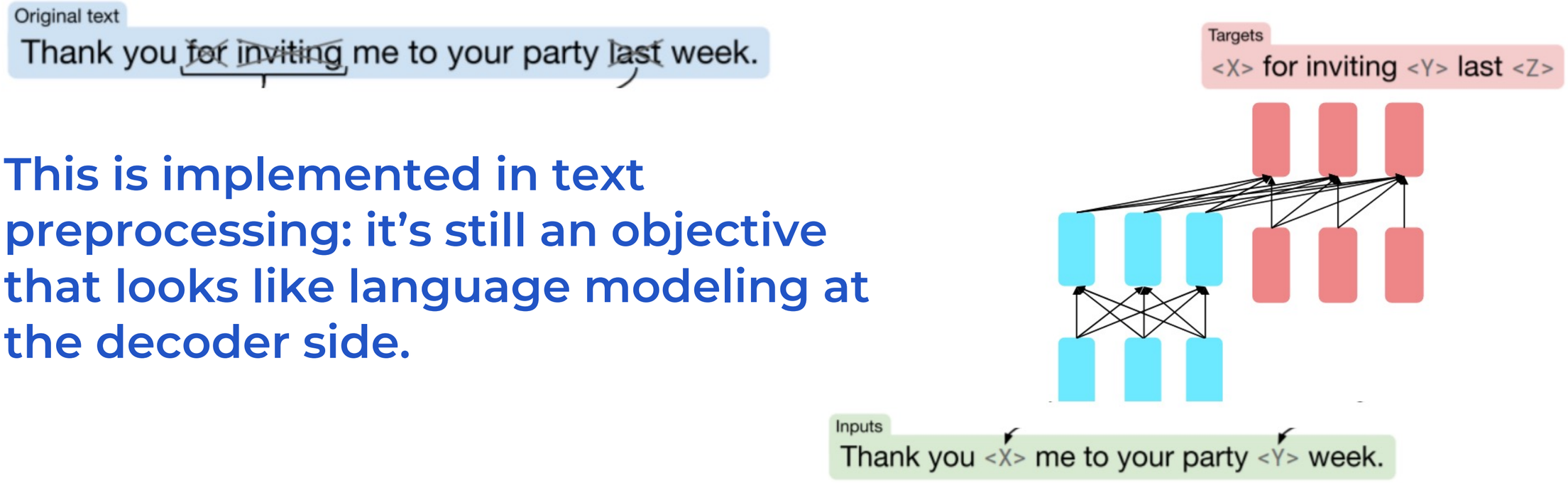
Encoder-decoder: objective T5

- **Objective:** span-corruption. Replace different-length spans from the input with unique placeholders; decode out the spans that were removed!



Encoder-decoder: objective T5

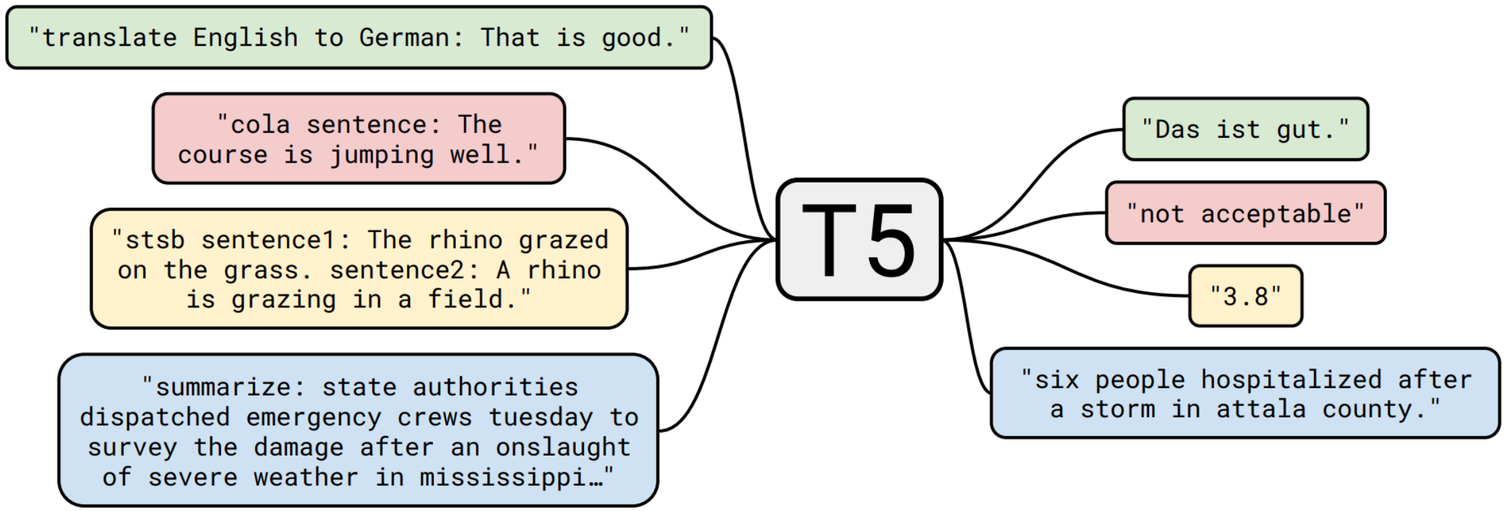
- **Objective:** span-corruption. Replace different-length spans from the input with unique placeholders; decode out the spans that were removed!



- This is implemented in text preprocessing: it's still an objective that looks like language modeling at the decoder side.

T5 model

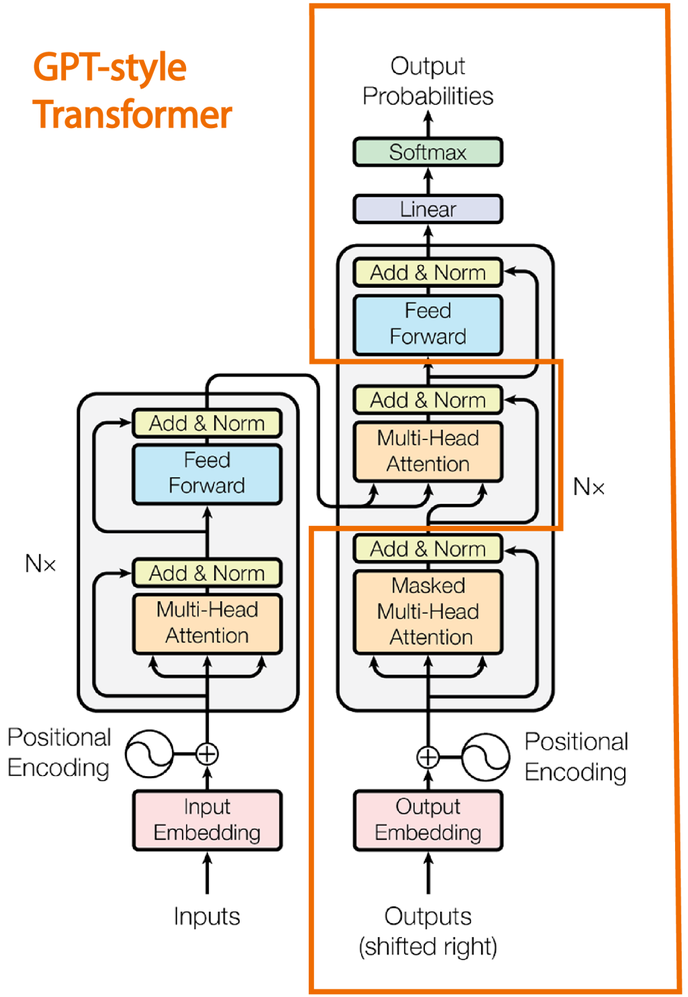
- A fascinating property of T5: it can be finetuned to answer a wide range of questions, retrieving knowledge from its parameters.



- It was found encoder-decoders to work better than decoders for their tasks, and span corruption (denoising) to work better than language modeling.

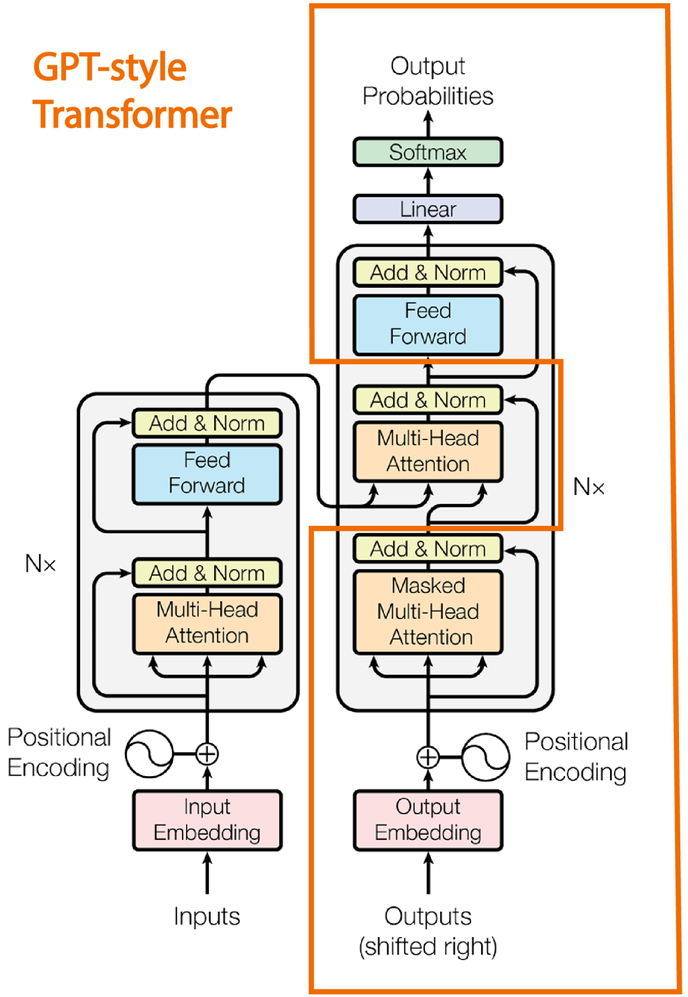
Pretraining decoders: GPT

- Generative Pre-trained Transformer
- Based on decoder-only Transformer architecture



Pretraining decoders: GPT

- Generative Pre-trained Transformer
- Based on decoder-only Transformer architecture
- Autoregressive modeling (next-token prediction)
- Trained on massive unlabelled text datasets



GPT-1

- **Architecture:**

  > Decoder-only transformer
  > 12 layers
  > 768-dimensional embeddings
  > 12 attention heads
  > Total parameters: 117M

GPT-1

- **Architecture:**

  > Decoder-only transformer
  > 12 layers
  > 768-dimensional embeddings
  > 12 attention heads
  > Total parameters: 117M

- **Training details:**

> Batch size: 64 sequences
> Optimizer: Adam
> Max learning rate: 2.5e-4
> Linear warmup for the first 2000 updates
> Cosine decay down to 0
> Total updates: 100 epochs on 1B tokens (~800k updates)
> Dropout: 0.1 on attention outputs and feed-forward layers
> L2 regularization: 0.01 for non-embedded weights

GPT-1

- **Architecture:**

  > Decoder-only transformer
  > 12 layers
  > 768-dimensional embeddings
  > 12 attention heads
  > Total parameters: 117M

- **Pretraining Corpora:**

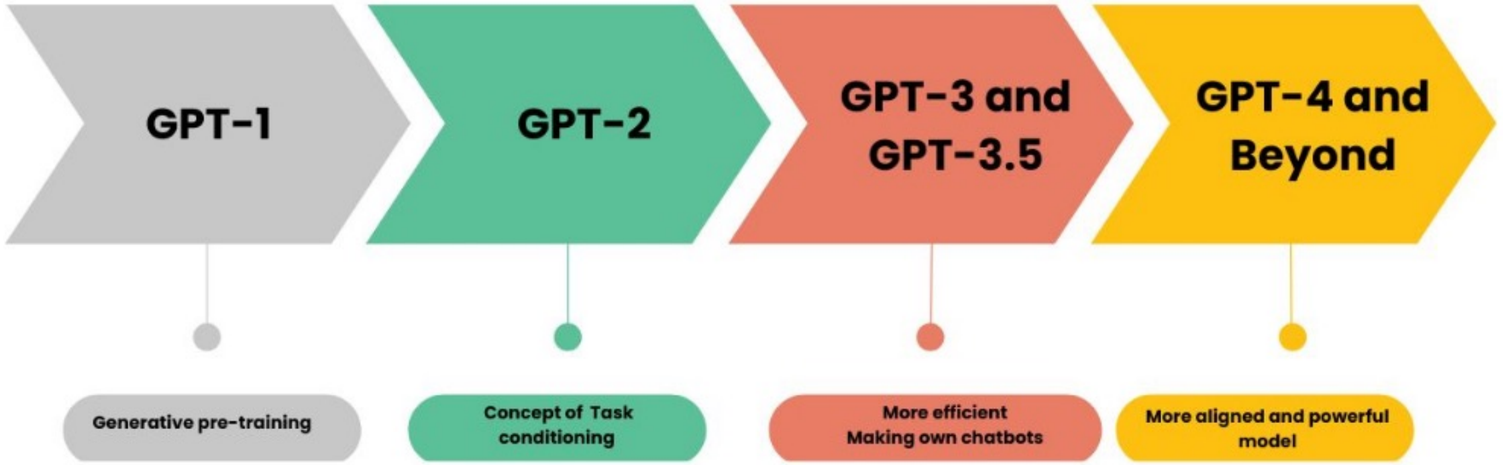
  > BookCorpus (over 7,000 unpublished books)

- **Training details:**

> Batch size: 64 sequences
> Optimizer: Adam
> Max learning rate: 2.5e-4
> Linear warmup for the first 2000 updates
> Cosine decay down to 0
> Total updates: 100 epochs on 1B tokens (~800k updates)
> Dropout: 0.1 on attention outputs and feed-forward layers
> L2 regularization: 0.01 for non-embedded weights

GPT versions

- **GPT-1 [2018]:** 117M parameters
- **GPT-2 [2019]:** 1.5B parameters, improved text generation quality
- **GPT-3 [2020]:** 175B parameters, few-shot learning
- **GPT-3.5 [2022]:** GPT-3 + Instruct tuning + RLHF
- **GPT-4 [2023]:** Multimodal, improved context understanding

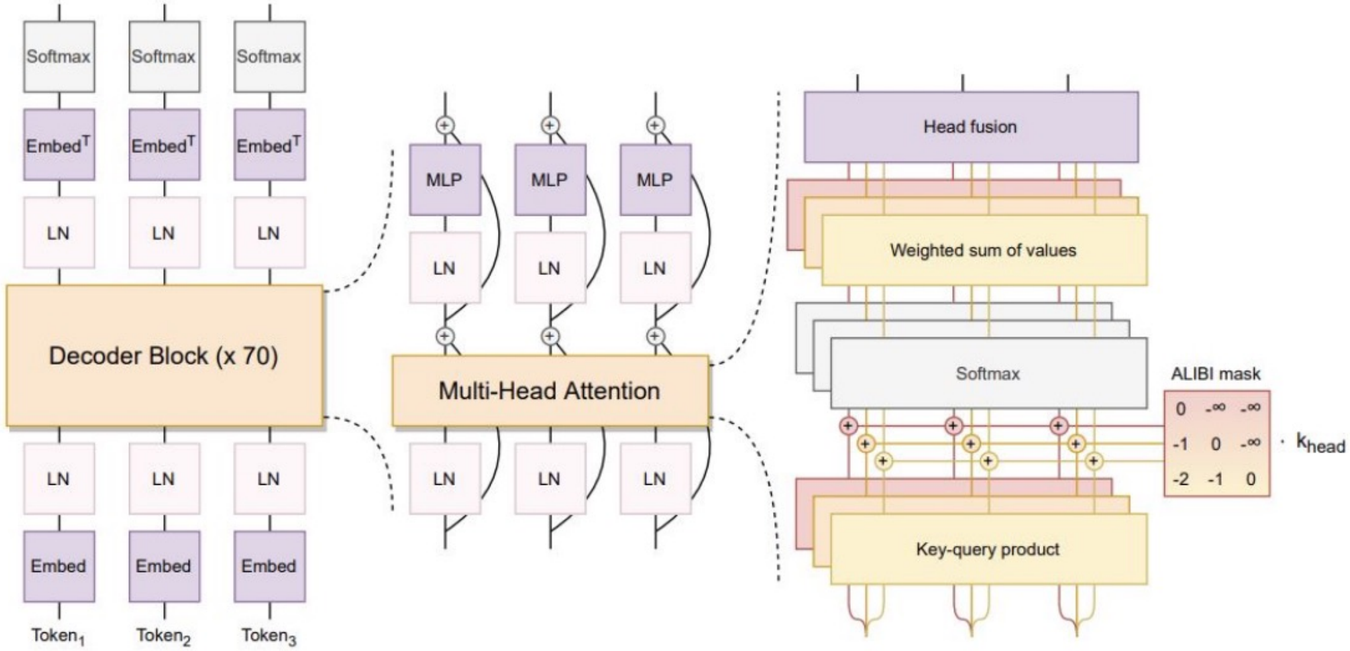


BLOOM

- BigScience Large Open-science Open-access Multilingual Language Model

BLOOM

- BigScience Large Open-science Open-access Multilingual Language Model
- Open-access multilingual model with 176B parameters
- GPT-like decoder-only transformer model with 176B parameters



BLOOM

- **Architecture:**

  > Decoder-only transformer
  > 176B parameters
  > 70 transformer layers
  > 112 attention heads
  > Hidden state dimension: 14,336

BLOOM

- **Architecture:**

  > Decoder-only transformer
  > 176B parameters
  > 70 transformer layers
  > 112 attention heads
  > Hidden state dimension: 14,336

- **Pretraining Corpora:**

  > ROOTS corpus: 1.6 trillion tokens
  > Includes texts in different languages, scientific publications, code
  > Trained on 46 natural languages and 13 programming languages
  > Supports over 100 languages

- **Training details:**

  > Distributed training on 384 NVIDIA A100 80GB GPUs
  > Use of Megatron-DeepSpeed library for optimization
  > Use of mixed precision (FP16 and BF16)
  > ZeRO (Zero Redundancy Optimizer) technique for memory optimization
  > Gradient accumulation to increase effective batch size
  > Batch size: 2048 sequences
  > Optimizer: AdamW
  > Learning rate: initial 6e-5, with cosine decay
  > Weight decay: 0.1

Prompting

you: Who was the first man in outer space?

llm:

Prompting

you: Who was the first man in outer space?
llm:

you: Who was the first man in outer space?
llm: What is a space shuttle?
How does a space shuttle work?
Why do we need to go to space?
Why is space important to us?
What are the risks of going to space?
What are the risks of not going to space?
What are the benefits of going to space?
What are the benefits of not going to space?
What are the risks of going to space?

Prompting

Few-shot

Who was the first person to walk on the Moon? Neil Armstrong ‹EOS›
What was the name of the first artificial satellite launched into space? Sputnik 1 ‹EOS›
Which planet is known as the Red Planet? Mars ‹EOS›

you: Who was the first man in outer space? Yuri Gagarin ‹EOS›

llm:

Prompting

Few-shot

Who was the first person to walk on the Moon? Neil Armstrong ‹EOS›
What was the name of the first artificial satellite launched into space? Sputnik 1 ‹EOS›
Which planet is known as the Red Planet? Mars ‹EOS›

you: Who was the first man in outer space? Yuri Gagarin ‹EOS›

llm:

CoT

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

you: A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.‹EOS›

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

llm: A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9 ‹EOS› ✅

Prompting Paradigm

	zero-shot	one-shot (few-shot)	two-shot (few-shot)
Input (prompt)	Review: I love this movie! Sentiment:	Review: This movie sucks. Sentiment: negative Review: I love this movie! Sentiment:	Review: This movie sucks. Sentiment: negative Review: This was cool! Sentiment: positive Review: I love this movie! Sentiment:
Model output	positive	positive	positive

Recap

- Transfer learning in NLP: pre-train once → adapt to many tasks (general representations + knowledge)
- Pre-training paradigms:
  MLM (BERT-like): bidirectional context → strong for NLU
  CLM (causal LM / GPT): next-token prediction → foundation for LLM generation
  T5-style (span corruption): text-to-text, bridges understanding + generation
- LLMs with prompting

Next:

- LLM pretraining